Practical 8

Aim: Implement an Assembly Code Generator: Implement the getreg() function to allocate registers to variables in the generated three-address code. (Use C/C++ language.)

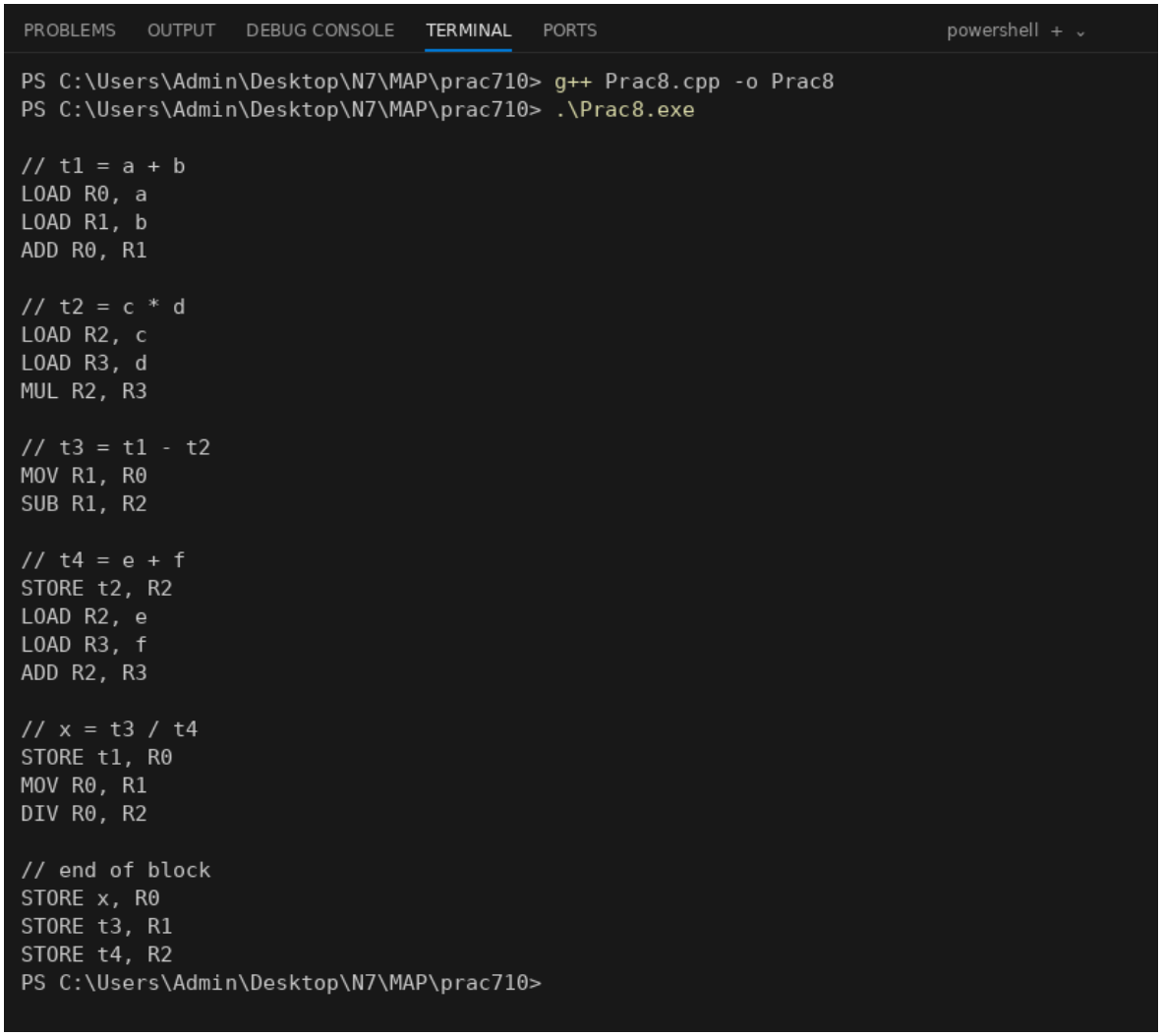
Prac8.cpp:

```
#include <iostream>
#include <string>
using namespace std;

string reg[4];
bool dirty[4];
int next_reg = 0;

int findReg(string v)
{
    for (int i = 0; i < 4; i++)
        if (reg[i] == v)
            return i;
    return -1;
}

int getreg(int avoid1, int avoid2)
{
    for (int i = 0; i < 4; i++)
        if (reg[i] == "" && i != avoid1 && i != avoid2)
            return i;

    while (next_reg == avoid1 || next_reg == avoid2)
        next_reg = (next_reg + 1) % 4;
    int r = next_reg;
    next_reg = (next_reg + 1) % 4;

    if (dirty[r])
        cout << "STORE " << reg[r] << ", R" << r << endl;
    reg[r] = "";
    dirty[r] = false;
    return r;
}

void load(int r, string v)
{
    int k = findReg(v);
    if (k != -1)
        cout << "MOV R" << r << ", R" << k << endl;
    else
    {
        cout << "LOAD R" << r << ", " << v << endl;
        reg[r] = v;
    }
}

int main()
{
    string x[] = {"t1", "t2", "t3", "t4", "x"};
    string y[] = {"a", "c", "t1", "e", "t3"};
    string sym[] = {"+", "*", "-", "+", "/"};
    string op[] = {"ADD", "MUL", "SUB", "ADD", "DIV"};
    string z[] = {"b", "d", "t2", "f", "t4"};
```

```
    for (int i = 0; i < 5; i++)
    {
        cout << "\n// " << x[i] << " = " << y[i] << " " << sym[i] << " " << z[i] << endl

        int r = getreg(findReg(y[i]), findReg(z[i]));
        load(r, y[i]);

        int rz = findReg(z[i]);
        if (rz == -1)
        {
            rz = getreg(r, -1);
            load(rz, z[i]);
        }
        cout << op[i] << " R" << r << ", R" << rz << endl;

        reg[r] = x[i];
        dirty[r] = true;
    }

    cout << "\n// end of block" << endl;
    for (int i = 0; i < 4; i++)
        if (dirty[i])
            cout << "STORE " << reg[i] << ", R" << i << endl;
    return 0;
}
```

Output:

Four registers R0 to R3 are used. Each TAC statement is shown as a comment followed by its pseudo assembly.



When all four registers are full, getreg() selects a victim in round robin order. If the victim holds a modified variable, it is saved first with STORE (e.g. STORE t2, R2). All remaining dirty values are stored at the end.